

Instance Based Learning:

K-Nearest Neighbours Learning:

* It's an approach to instance based learning

* All the instance correspond to points in the n -dimensional space R^n .

* The nearest neighbors of an instance are defined in terms of the standard Euclidean distance.

* The arbitrary instance x be described by the feature vectors.

$$(a_1(x), a_2(x), \dots, a_n(x)).$$

$a_r(x)$ denotes the value of the r th attribute of instance x .

* Then the distance between 2 instances x_i & x_j is defined to be $d(x_i, x_j)$, where

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^n (a_r(x_i) - a_r(x_j))^2}$$

* In nearest-neighbor learning the target function may be either discrete-valued or real-valued.

* Let us first consider learning discrete-valued target functions of the form.

$$f: R^n \rightarrow V$$

↳ feature space

where V is the finite set $\{V_1, \dots, V_s\}$

* The K-nearest neighbor algorithm for approximating a discrete-valued target function is given below:

Classification
Training algorithm.

$$\hat{f}(x, q) = \underset{r \in V}{\operatorname{argmax}} \sum_{i=1}^K \delta(r, f(x_i))$$

Example

K=3

Sl No	Height	Weight	Target	Distance	
1	150	50	medium	8.06	1
2	155	55	medium	2.24	3
3	160	60	Large	6.71	2
4	161	59	Large	6.40	
5	158	65	Large	11.05	
6	157	54	? L		

$$1. \sqrt{(150-157)^2 + (50-54)^2} = \sqrt{49+16}$$

$$\begin{aligned} \hat{f}(x_q) &= \delta(m, f(x_1)) + \delta(m, f(x_2)) + \delta(m, f(x_3)) \\ &= \delta(m, m) + \delta(m, L) + \delta(m, L) \\ &= 1 + 0 + 0 = 1 \end{aligned}$$

$$\begin{aligned} \hat{f}(x_q) &= \delta(L, f(x_1)) + \delta(L, f(x_2)) + \delta(L, f(x_3)) \\ &= \delta(L, m) + \delta(L, L) + \delta(L, L) \\ &= 0 + 1 + 1 = 2 \end{aligned}$$

New Instance

157

54

L

Real Valued Function

* It is given as $f: R^n \rightarrow R$

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^K f(x_i)}{K} \quad K=3$$

S/No	Height	Weight	Target	Distance	Nearest point
1	150	50	1.5	8.06	1
2	155	55	1.2	2.24	3
3	160	60	1.8	6.71	2
4	161	59	2.1	6.40	
5	158	65	1.7	11.05	
6	157	54	1.7		

$\hat{f}(x_q) = \frac{1.8 + 1.2 + 2.1}{3} = \frac{5.1}{3} = 1.7$

Distance weighted K-nearest neighbor Algorithm

Discrete Valued Function

$$\hat{f}(x_q) \leftarrow \underset{y \in Y}{\operatorname{argmax}} \sum_{i=1}^K w_i \delta(y, f(x_i))$$

$$w_i = \frac{1}{d(x_q, x_i)^2}$$

$$\delta(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{if } a \neq b \end{cases}$$

S/No	Height	Weight	Target	Distance	Distance	Nearest
1	150	50	medium	8.06		
2	155	55	medium	2.24	0.45	3
3	160	60	Large	6.71	0.15	1
4	161	59	Large	6.40	0.16	2
5	158	65	Large	11.05		
6	157	54	1.7			

$\frac{1}{(2.24)^2} =$

$\uparrow$ medium

$$\hat{f}(x_q) = 0.45d(m,m) + 0.15s(m,L) + 0.16s(m,L)$$

$$= 0.45 * 1 + 0.15 * 0 + 0.16 * 0$$

$$= 0.45$$

$$\hat{f}(x_q) = 0.45s(L,m) + 0.15d(L,L) + 0.16s(L,L)$$

$$= 0.45 * 0 + 0.15 * 1 + 0.16 * 1$$

$$= 0 + 0.31$$

$$= 0.31$$

Real Valued Target Function.

$$\hat{f}(x_q) = \frac{\sum_{i=1}^K w_i f(x_i)}{\sum_{i=1}^K w_i}$$

where

$$w_i = \frac{1}{d(x_q, x_i)^2}$$

Sl No	Height	Weight	Target	Distance	$\frac{1}{\text{Distance}}$
1	150	50	1.5	8.06	
2	155	55	1.2	2.24	0.45
3	160	60	1.8	6.71	0.15
4	161	59	2.1	6.40	0.16
5	158	65	1.7	11.05	
6	157	54	<u>?</u> 1.51		

$$\hat{f}(x_q) = \frac{0.45 * 1.2 + 0.15 * 1.8 + 0.16 * 2.1}{0.45 + 0.15 + 0.16}$$

~~2.24~~

$$= 1.146 / 0.76 = 1.51$$

Locally Weighted Regression.

* Locally weighted regression is instance based learning algorithm.

Training algorithm:

For each training example $(x, f(x))$, add the example to the list training examples.

Classification algorithm:

Given a query instance x_q to be classified.

- Let $x_1, \dots, x_k$ denote the k instance from training examples that are nearest to x_q .
- Return.

$$\hat{f}(x_q) \leftarrow \underset{v \in V}{\operatorname{argmax}} \sum_{i=1}^k \delta(v, f(x_i))$$

Where $\delta(a, b) = 1$ if $a = b$ and where $\delta(a, b) = 0$ otherwise.

Locally Weighted Regression:

- * It is an instance based learning algorithm.
- * It is a memory-based method.
- * There are 2 methods of learning
 - Model-based Ex: Neural Network.
 - Memory-based Ex: LWR
- * LWR is a memory-based method that perform a regression around a point of interest using only training data that are 'local' to that point.
- * Local - because the function is approximated based only on data near the query point.
- * Weighted - because the contribution of each training example is weighted by its distance from the query point.
- * Regression - used for the community for the problem of approximating real-valued function.
- * Given a new query instance x_q , the general approach in locally weighted regression is to construct an approximation $\hat{f}$ that fits the training examples in the neighborhood surrounding x_q .

Working:

- * Consider locally weighted Regression in which the target function 'f' is approximated near x_q using linear function of the form.

$$\hat{f}(x) = w_0 + w_1 a_1(x) + \dots + w_n a_n(x)$$

where $a_i(x)$ are the attributes of the instance 'x'

w_n the coefficient of the attribute.
also called the weight of the attribute.

- * Since we do not know the weights w_i values. we choose it randomly and then we fine tune the values in a such a way that this function will fit to the training example.

- * To fine tune / modify the weights we use Gradient descent of ANN

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

where:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta \sum_{d \in D} (t_d - o_d) x_{id}$$

- * The same approach is used in LWR.

* Derived methods are used to choose weights minimize the squared error summed over the set D of training examples using gradient descent

$$E = \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2$$

$\downarrow$ actual value $\downarrow$ calculated value

* Which led us to the Gradient descent training rule.

$$\Delta w_j = \eta \sum_{x \in D} (f(x) - \hat{f}(x)) a_j(x)$$

* Need to modify this procedure to derive a local approximation rather than a global one.

* This can be done by refining the error criterion 'E' to emphasize fitting the local training example.

* 3 possible criteria can be considered.

1. Minimize the squared error over just the k -nearest neighbors.

$$E_1(x_q) = \frac{1}{2} \sum_{x \in K \text{ nearest neighbor of } x_q} (f(x) - \hat{f}(x))^2$$

ie rather than considering all the training example consider only the k NN and reduce the error.

2. minimize the squared error over the entire set D of training examples, while weighting the error of each training example by some decreasing function K of its distance from x_q : (minimum distance more important)
decreasing function.

$$E_2(x_q) = \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2 K(d(x_q, x))$$

↓
Distance of new instance from the current point

3. Combine 1 & 2

$$E_3(x_q) = \frac{1}{2} \sum_{x \in K \text{ nearest nbrs of } x_q} (f(x) - \hat{f}(x))^2 K(d(x_q, x))$$

* If we choose criterion three, we derive the Gradient descent rule, we obtain the following training rule.

$$\Delta w_j = \eta \sum_{x \in K \text{ nearest nbrs of } x_q} K(d(x_q, x)) (f(x) - \hat{f}(x)) q_j(x)$$

Radial Basis Functions: distance weighted regn

* It is closely related to LWR & also to ANN.

* In RBF, the learned ~~function~~ hypothesis is a function of the form. distance function.

$$\hat{f}(x) = w_0 + \sum_{u=1}^k w_u \left(K_u(d(x_u, x)) \right)$$

where. x_u is an instance from 'x'
 $K_u(d(x_u, x))$ - kernel function
 it decreases as the distance $d(x_u, x)$ increases.

$k \rightarrow$ user provided constant which specifies the no of kernel funcⁿ to be included.

* Even though $\hat{f}(x)$ considers a global approximation to $f(x)$, the contribution from each of the $K_u(d(x_u, x))$ terms is localized to a region near by the point x_u .

$f(x)$

Case Based Reasoning:

* It is one of the instance based learning.

* The instance based have 3 properties

* They are lazy learners

* Classification is different for each instance

* Instances are represented with 'n' dimensional euclidean space.

* In CBR

Everything is considered as case & based on previous cases we propose a solution.

In case base reasing, instance are represented as symbols (not value)

CBR has 3 components.

1. Similarity Functions or distance measures.
2. Approximation / Adjustment of instances
3. Symbolic representation of instances.

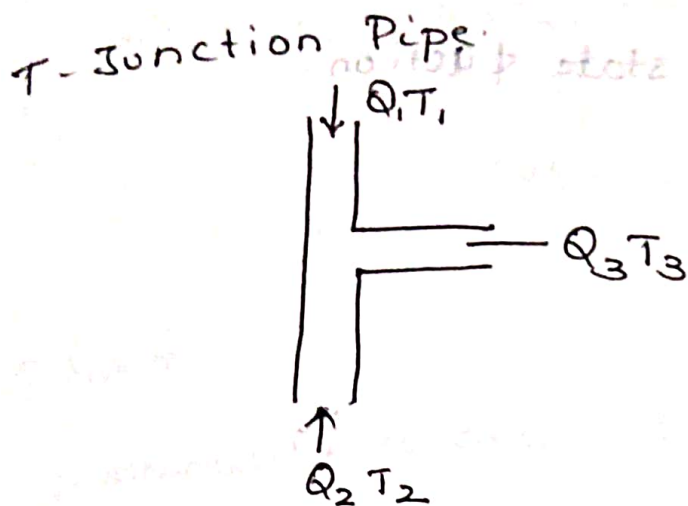
* For modeling CBR we use 'CADET' system.

CADET - Case based design Tool.

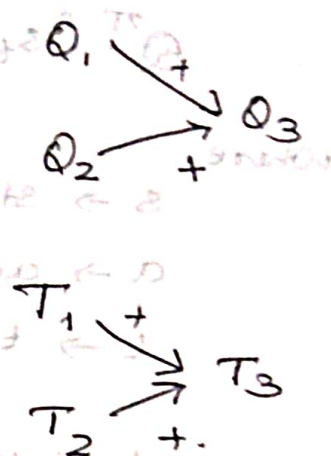
* In CADET system we already have 75 predefined libraries.

* Using these libraries new cases can be designed.

Example: Modern water taps



functions



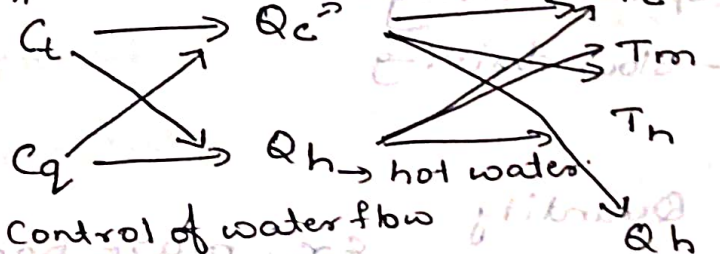
Q means water flow.

T means Temperature

Requirement.

Another tap: control temp & water flow

control of temp



Temperature cold

Temperature medium

Temperature ~~low~~ high

Control of water flow

Instance based methods such as K-nearest neighbour

LWR has 3 properties.

1. Lazy Learning methods

2. Classify new query instances by analysing similar instances while ignoring instances that are very different from the query.

3. They represent instances as real valued points in an n -dimensional Euclidean space.

CBIR is a learning algorithm based on the first principle, but not the third.

Q-Learning.

① Policy - combination of state & action.

$$Q^{\pi}(s_t, a_t)$$

where

$s \rightarrow$ state.

$a \rightarrow$ action

$t \rightarrow$ time

Policy is undertaken by agent.

Agent observes the given state & selects best possible action.

Ex: empty road - speed driving
crowdy Road - slow driving.

② Reward : Scalar Quantity
correct action

Ex: maze game.

③ Penalty : -ve reward.
wrong action.

Ex: parking in no parking zone.

Q-learning mainly depends on 2 factors.

① Q-function

② Q-Table.

Q-Function.

$$Q^{\pi}(s_t, a_t) = E(R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n R_{t+n} | s_t, a_t)$$

discount factor.

1) Bellman Equation.

Q-table:

combination of action & states

Ex: In a Game $\leftrightarrow$

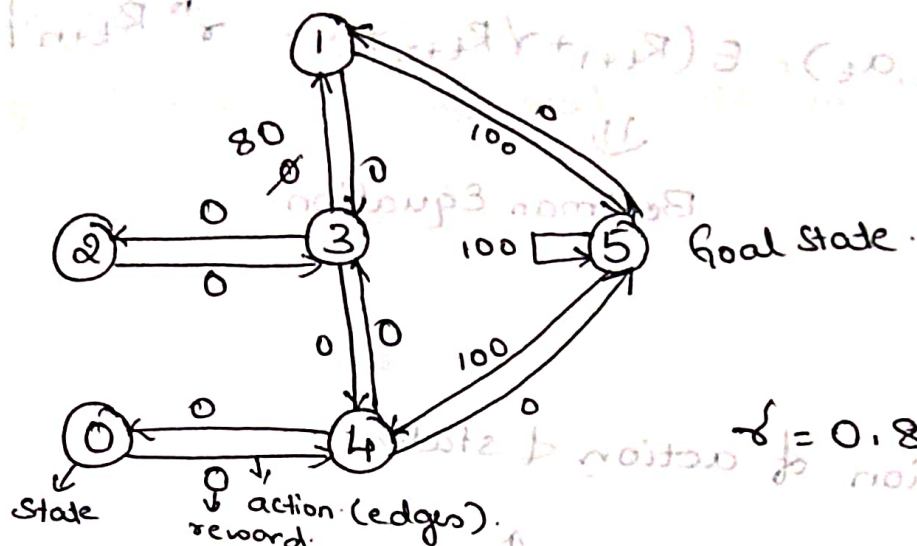
Action - Up, down, left, right.

State: Start, end, Reward, health etc.

Step follow:

- ① Exploration: explore all the possible path
- ② Exploitation: best possible path is identified by removing redundant, unwanted path.
- ③ Initialize Q table. $\Rightarrow 0$
- ④ Choose Action.
- ⑤ Perform Action.
- ⑥ Measure ~~Action~~ Reward (+ve / -ve)
- ⑦ Update the Q-table.

Ex:



$$\gamma = 0.8$$

$$S = 3,$$

$$A = 2, 1, 4,$$

$$r = 0$$

$$S' = 1$$

$$\hat{Q}(3, 1) = 0 + 0.8 \max_a (0, 100) = 80$$

$$\hat{Q}(3, 2) = 0 + 0.8 \max_a (0, 0) = 0$$

~~Q(3, 4) = 0 + 0.8 \max_a (0, 0) = 0~~

$$S = 3, 1$$

$$A = 5, 3, 2, 1, 4.$$

$$r = 0, 100$$

$$S' = 1, 5$$

$$\hat{Q}(1, 5) = 100 + 0.8 \max_a (0, 0) = 100$$

$$100 + 0.8 \max_a (0, 0) = 100$$

$$= 100 + 0.8 \max_a (0, 0) = 100$$

Bmi	Age	Sugar.
33.6	50	1
26.6	30	0
23.4	40	0
43.1	67	0
35.3	23	1
35.9	67	1
36.7	45	1
25.7	46	0
23.3	29	0
31	51	1

$K=3$

Test Bmi = 23.6 , Age = 40 sugar = ?

x_1	x_2	o/p
7	7	0
7	4	0
3	4	1
1	4	1

3, 7 $K=3$